

IOTA Service ↔ Backend Java Integration Study

Document de passation technique entre le service IOTA indépendant (TypeScript/Node.js) et le Backend Java H2S (Quarkus) — état réel du code, comparaison, et préparation de l'intégration.

Auteur	Hicham Ait El Aouri
Date	22 septembre 2026
Projet 1	java_backend_H2S — GitLab platform.git, branche dev
Projet 2	iota-service — TypeScript/Node.js, indépendant

DRAFT · FOR REVIEW

Page de garde

Projet H2S

IOTA Service ↔ Backend Java Integration Study

Document de passation technique — à destination du responsable Backend Java et de l'équipe H2S.

- **Auteur** : Hicham Ait El Arouri
- **Date** : 22 septembre 2026
- **Statut** : DRAFT / FOR REVIEW
- **Portée** : `iota-service` (TypeScript/Node.js, indépendant) ↔ `java_backend_H2S` (Quarkus, monorepo équipe, branche `dev`, GitLab `platform.git`)

Ce document est un document de travail technique. Il ne constitue **pas** une décision d'architecture finale. Chaque affirmation est marquée d'un statut : `[EXISTANT]`, `[IMPLÉMENTÉ]`, `[PARTIEL]`, `[NON IMPLÉMENTÉ]`, `[NON TROUVÉ]`, `[PROPOSITION]`, `[À VALIDER]`, `[À VÉRIFIER]`. Toute affirmation non vérifiable dans le code, les tests ou une exécution réelle est explicitement marquée comme telle — jamais présentée comme un fait acquis.

Table des matières

1. Executive Summary
2. H2S Context
3. Current Java Backend Architecture
4. Current `iota-connector`
5. Current `iota-service`
6. Work Already Completed by Hicham
7. Work Already Existing in Java Backend
8. Java Local Ledger vs Real IOTA
9. Detailed Comparison
10. Responsibilities
11. Integration Readiness
12. First Java → IOTA Integration Milestone
13. Target Architecture
14. REST Contract
15. RequestId and Idempotence
16. Security
17. Data Persistence
18. On-chain / Off-chain
19. Reconciliation
20. Digital Twin Integration
21. Energy Event Integration
22. Tokenization / Certificates / Move
23. Required Changes in Java
24. Required Changes in `iota-service`
25. Git / Repository Strategy
26. Migration Roadmap
27. Risks
28. Open Questions
29. Validation Checklist
30. Final Summary

1. Executive Summary

Le projet H2S dispose aujourd'hui de **deux travaux distincts et non connectés** :

1. `java_backend_H2S` — un monorepo Quarkus 3.39.2 (Java 21) de l'équipe, en migration progressive (strangler-fig) d'un monolithe vers 16 microservices, dont `iota-connector` et `digital-twin`. Ce backend possède une plateforme d'échange énergétique **métier complète et mature** (marketplace, wallets EUR, settlement, certificats verts, registre local signé, idempotence, réconciliation). `[EXISTANT – confirmé par le code]`

2. **iota-service** — un microservice TypeScript/Node.js indépendant développé par Hicham, avec une **connexion réelle et testée au SDK @iota/iota-sdk** sur le réseau **Devnet** IOTA Rebased : wallets (keypair Ed25519 réel), transactions natives signées et exécutées on-chain, idempotence persistante PostgreSQL, réconciliation lecture-seule. **[EXISTANT – confirmé par le code et les tests]**

Fait central de ce document, vérifié ligne à ligne dans les deux codebases : aucun appel réel n'existe aujourd'hui entre ces deux projets. **iota-connector** (Java) n'a aucune classe, aucune configuration, aucune URL référençant **iota-service**. Le principe d'architecture **Backend Java → REST/JSON → IOTA Service → @iota/iota-sdk → IOTA Rebased** discuté avec l'équipe est une **cible à étudier**, pas un état réalisé. **[NON IMPLÉMENTÉ]**

Deuxième fait central : le Backend Java possède déjà un mécanisme qu'il faut se garder de confondre avec une intégration IOTA réelle — un **registre local** (**Ledger** / **LedgerService**), qui chaîne des hachés SHA-256 signés Ed25519 en PostgreSQL. Il produit une **preuve cryptographique locale**, pas une transaction sur le réseau IOTA. Le code d'ancrage distant vers un nœud IOTA (**AncrageIotaHttp**) existe mais est **désactivé par défaut** et n'a jamais été exercé contre un vrai nœud dans ce dépôt. **[IMPLÉMENTÉ localement – NON IMPLÉMENTÉ au sens IOTA réel]**

Ce document a pour objectif de donner au responsable Backend Java tout ce qu'il faut pour comprendre les deux travaux sans devoir réanalyser l'un ou l'autre projet, et de préparer — sans la décider — l'intégration entre les deux.

2. H2S Context

H2S est une plateforme de gestion énergétique (production, consommation, stockage batterie, marketplace P2P, certificats de garantie d'origine) reposant sur :

- Un **Digital Twin batterie** (**services/digital-twin**) : estimation de SoC par filtre de Kalman étendu + modèle Thévenin 2RC, décisions de charge/décharge, validation par un moteur de règles séparé (**rule-engine**). **[EXISTANT – confirmé par le code]**
- Une **plateforme d'échange énergétique** (**services/iota-connector**, brique 8) : agents de marché, enchères, règlements, portefeuilles EUR, certificats verts, registre de preuve. **[EXISTANT – confirmé par le code]**
- Une architecture Quarkus en **migration progressive** (strangler-fig) depuis un monolithe historique (**backend/**, package **com.h2s.***) vers 16 microservices extraits, derrière une passerelle interne (**gateway/**). **[EXISTANT – confirmé par le code]**

Le nom « IOTA » apparaît largement dans le code et la documentation du Backend Java (noms de classes, de tables, de configuration) — **ce document distingue systématiquement l'usage du mot « IOTA » comme vocabulaire de domaine métier (registre, preuve, wallet interne) de l'usage réel du réseau IOTA Rebased**, conformément à la règle NO BLUFF de ce projet.

Un échange récent avec l'équipe H2S a confirmé que le responsable Backend Java est prêt à intégrer **iota-service** dès que le contrat REST sera documenté — c'est l'objet de la Section 14. Un troisième acteur métier, le **stockage**, a été évoqué pour la marketplace énergétique (négociation d'un prix de stockage entre producteur et consommateur) ; ses implications fonctionnelles ne sont pas encore précisées par l'équipe. **[À VALIDER]** — ce document ne modifie aucune décision d'architecture IOTA sur cette seule base et ne suppose rien de son comportement.

3. Current Java Backend Architecture

[EXISTANT – confirmé par le code, audit read-only complet]

- **Framework** : Quarkus 3.39.2, Java 21, Maven multi-module. **Pas Spring Boot**.
- **Repository** : GitLab <https://gitlab.com/h2s-project/poc-apartment/platform.git>, branche **dev**.
- **Layout** : monorepo **apps/projeth** (plateforme H2S) + **apps/website** + **infra/**.
- **Migration en cours (strangler-fig)** : un monolithe historique (**backend/**, ~440 routes API) est décomposé progressivement en **16 microservices extraits** : **agent-sync**, **auth**, **billing**, **commercial**, **contracts**, **digital-twin**, **edge-orchestrator**, **enedis-connector**, **iota-connector**, **marketing**, **portfolio**, **reporting**, **rte-connector**, **rule-engine**, **sig**, **support-chat**, **users**. À date de cet audit, 126/440 routes API restent servies uniquement par le monolithe.
- **Passerelle interne** (**gateway/**, alias **h2s-passerelle**) : routage statique par préfixe le plus long. Route **/api/exchange** → **iota-connector** ; **/api/battery** → **digital-twin**.
- **Bibliothèques partagées** : **h2s-noyau** (fondations, **RequestIds**), **h2s-securite** (identité interne signée Ed25519), **h2s-architecture** (règles ArchUnit — interdiction de **@Entity** dans les services extraits), **h2s-evenements** (pattern outbox transactionnel — **pas** Kafka/RabbitMQ).
- **Base de données** : PostgreSQL 16 + PostGIS 3.4, migrations SQL versionnées manuelles (**Vn__description.sql**, **V1** à **V150**), non-Flyway/Liquibase historiquement, script maison **apply.sh**). 129+ tables **@Entity** côté monolithe ; microservices extraits utilisent des POJOs purs + mapping Hibernate XML (**orm-<service>.xml**), **jamais @Entity** (règle ArchUnit).
- **Sécurité** : Keycloak/OIDC pour les utilisateurs finaux ; identité interne signée Ed25519 propagée par la passerelle entre services ; secret partagé (**X-H2S-Cle-Interne**) pour les routes **/interne/***.
- **MQTT** : Vert.x MQTT (**io.vertx:vertx-mqtt**), utilisé par **agent-sync** (canal agent↔cloud EMQX) et **rte-connector** / **backend** (signaux RTE). Pas de **org.eclipse.paho** côté Java.
- **IA/ML** : moteur Python séparé (**ai-engine/**, FastAPI, LightGBM), appelé en HTTP — non embarqué en Java.

4. Current iota-connector

[EXISTANT – confirmé par le code, lecture intégrale de apps/projeth/services/iota-connector/]

`iota-connector` (module Maven `h2s-iota-connector`, brique 8 du plan de migration) est un service Quarkus autonome, port `8091`, qui porte la plateforme d'échange énergétique H2S :

- Marché P2P (agents producteurs/consommateurs/électrolyseurs, tours d'enchère, appariement).
- Règlements financiers (`SettlementService`) et portefeuilles internes en euros (`WalletService`).
- Certificats verts (garantie d'origine), avec saga de vente en 5 temps (`CertificatSaga`).
- Jetons « kWh vert » (`TokenService`) — des lignes PostgreSQL, pas des tokens on-chain.
- Un registre de preuve local (`Ledger` / `LedgerService`) : chaîne de hachés SHA-256 signée Ed25519, stockée en PostgreSQL, avec verrou PostgreSQL global (`pg_advisory_xact_lock`) pour garantir une chaîne unique.
- Un point d'ancrage IOTA distant (`AncrageIotaHttp`) : **code réel, mais désactivé par défaut** (`h2s.ledger.mode=local` partout, `H2S_IOTA_NODE_URL` jamais renseigné dans le dépôt). [IMPLÉMENTÉ mais code mort en pratique]
- Un mécanisme d'idempotence et d'ancrage via outbox transactionnel (`AncrageEntrant`, `com.h2s:h2s-evenements`), utilisé par les autres services H2S pour faire signer une preuve.
- Sécurité : secret partagé (`GardeInterne`) + identité Ed25519 propagée depuis la passerelle (`EnTetesPropages`), pas d'authentification vers un système externe de type `iota-service`.

Aucune dépendance vers un SDK IOTA — le `pom.xml` du module le dit explicitement dans son commentaire d'en-tête (« Aucune dépendance IOTA... embarquer un SDK Java non verrouillé aurait figé un choix que le client n'a pas encore tranché »). Le détail exhaustif classe par classe est en Section 9 et dans l'audit dédié déjà produit (annexe technique disponible séparément).

5. Current iota-service

[EXISTANT – confirmé par le code, lecture intégrale de `src/`, `migrations/`, `tests/`, `package.json`]

`iota-service` est un microservice TypeScript/Node.js indépendant, sans framework HTTP (`node:http` natif), utilisant le SDK officiel `@iota/iota-sdk@1.15.1` et `pg@8.23` pour PostgreSQL. Port par défaut **4100**.

Inventaire (chemins exacts)

Élément	Chemin	Rôle	État
Entrée	<code>src/index.ts</code>	Démarré migrations, construit le repository PostgreSQL, lance le serveur HTTP	[IMPLÉMENTÉ]
Serveur/routage	<code>src/server.ts</code>	Routeur HTTP manuel, aucun framework	[IMPLÉMENTÉ, TESTÉ]
Config	<code>src/config/index.ts</code>	Port, réseau IOTA (devnet/testnet/mainnet), config DB ; garde-fou anti-Mainnet hors production	[IMPLÉMENTÉ]
Client IOTA	<code>src/iota/client.ts</code>	Singleton <code>IotaClient</code> du SDK officiel, <code>getFullnodeUrl(network)</code>	[IMPLÉMENTÉ]
Connectivité	<code>src/iota/connectivity.ts</code>	Vérifie la connexion réelle (lecture d'un checkpoint réseau)	[IMPLÉMENTÉ, TESTÉ]
Wallet store	<code>src/iota/wallet/walletStore.ts</code>	Persistance fichier JSON local en clair , <code>.dev-wallets/</code> — explicitement dev-only	[IMPLÉMENTÉ – NON PRODUCTION-READY, assumé dans le code]
Wallet manager	<code>src/iota/wallet/walletManager.ts</code>	Génération keypair Ed25519 réel, dérivation d'adresse IOTA, lecture de balance réseau	[IMPLÉMENTÉ, TESTÉ]
Wallet controller	<code>src/iota/wallet/walletController.ts</code>	Couche HTTP pure (validation, statuts)	[IMPLÉMENTÉ, TESTÉ]
Transaction service	<code>src/iota/transaction/transactionService.ts</code>	Construction PTB, signature, exécution réelle sur Devnet, réconciliation	[IMPLÉMENTÉ, TESTÉ]
Transaction repository	<code>src/iota/transaction/{transactionRepository,postgresTransactionRepository}.ts</code>	Idempotence atomique PostgreSQL (<code>INSERT...ON CONFLICT DO NOTHING</code>) + implémentation mémoire pour tests	[IMPLÉMENTÉ, TESTÉ]
Transaction controller	<code>src/iota/transaction/transactionController.ts</code>	Couche HTTP, codes de statut nuancés (200/201/202)	[IMPLÉMENTÉ, TESTÉ]

Élément	Chemin	Rôle	État
DB pool/migrations	<code>src/db/pool.ts</code> , <code>src/db/migrate.ts</code> , <code>migrations/0001_create_iota_transactions.sql</code>	Runner de migrations maison, forward-only, une seule table (<code>iota_transactions</code>)	[IMPLÉMENTÉ, TESTÉ]
Documentation contrat	<code>docs/integration-backend.md</code>	Contrat REST/JSON destiné à un Backend externe, déjà rédigé pour un lecteur non-TypeScript	[EXISTANT]
Tests	<code>tests/*.test.ts</code> (11 fichiers)	<code>node:test</code> , y compris un test contractuel HTTP pur (<code>contract.test.ts</code>)	[TESTÉ]

Ce que le service NE fait PAS (vérifié, pas supposé)

- Pas de tokenisation on-chain, pas de certificat, pas d'asset/objectId, pas de contrat Move. [NON IMPLÉMENTÉ]
- Pas de DID / W3C Verifiable Credentials. [NON IMPLÉMENTÉ]
- Pas d'authentification entrante (aucun middleware, `docs/integration-backend.md` §13 le confirme explicitement). [NON IMPLÉMENTÉ]
- Pas de wallet en PostgreSQL — un seul wallet store fichier, en clair, dev-only. [PARTIEL – dev only]
- Pas de réconciliation par lot (seulement unitaire, par `requestId`). [PARTIEL]
- Transfert de valeur réel **non démontré** — aucun wallet Devnet financé dans cet environnement. [À VÉRIFIER en conditions réelles]

6. Work Already Completed by Hicham

Écrit à la première personne, uniquement ce qui est vérifié.

J'ai développé un service IOTA indépendant en TypeScript/Node.js afin d'isoler la dépendance au SDK IOTA du Backend métier, suivant le principe « le Backend décide, le service IOTA exécute et fournit la preuve ».

Ce que j'ai réellement construit et vérifié :

- **Architecture** : `node:http` natif, sans framework, structuré en couches (config, client IOTA, wallet, transaction), chaque couche testable indépendamment.
- **Connexion réseau IOTA réelle** : `IotaClient` du SDK officiel `@iota/iota-sdk` , connecté au réseau **Devnet**, avec un garde-fou de code qui **interdit physiquement** Mainnet hors environnement de production (`config/index.ts:53-58`), conformément à une feuille de route validée (`ROADMAP_IOTA_H2S.md` , Devnet phases 2-8 → Testnet 9-12 → Mainnet 13+).
- **Wallets** : génération d'un vrai keypair Ed25519 (`Ed25519Keypair`), dérivation d'une vraie adresse IOTA, lecture de balance réelle depuis le réseau. Stockage aujourd'hui en fichier JSON local, **explicitement documenté comme non destiné à la production** (aucune clé privée ne sort jamais des réponses HTTP, vérifié par des tests automatisés).
- **Transactions** : construction d'un `Transaction` (PTB) réel via `splitCoins` / `transferObjects` , signature avec le keypair du wallet source, exécution réelle via `signAndExecuteTransaction` , interprétation du **vrai statut réseau** (`effects.status`), jamais déduit de l'absence d'exception.
- **Idempotence persistante** : une contrainte `UNIQUE` PostgreSQL et une opération atomique `INSERT...ON CONFLICT DO NOTHING RETURNING` garantissent qu'un `requestId` rejoué, même sous concurrence multi-instance, ne déclenche jamais deux fois l'opération blockchain.
- **Réconciliation** : une route dédiée compare l'état persisté en PostgreSQL à l'état réel observé sur IOTA, en lecture seule, avec une classification explicite de 5 cas (`coherent` , `inconsistent` , `not_found` , `unable_to_check` , `no_digest`) — dont une limite connue et documentée dans le code lui-même (un enregistrement `pending` sans digest n'est pas réconciliable si le service crashe entre diffusion et écriture).
- **REST** : 9 endpoints (`/health` , `/api/iota/network` , `/api/iota/wallets*` , `/api/iota/transactions*` , `/api/iota/reconcile`), formats d'erreur uniformes, codes de statut nuancés selon le résultat réel (pas seulement l'acceptation de la requête).
- **Tests** : 11 fichiers de tests (`node:test`), y compris une suite contractuelle (`tests/contract.test.ts`) qui exerce le service exclusivement via `fetch()` HTTP, sans importer de code interne — pensée pour représenter le point de vue d'un futur client Backend Java.
- **Documentation** : `docs/integration-backend.md` , rédigé spécifiquement pour un lecteur Backend qui n'a besoin de rien connaître à TypeScript/SDK/cryptographie.

Limites actuelles, non maquillées : wallet store dev-only (fichier, pas de coffre-fort), aucune authentification entrante, aucun transfert de valeur réel démontré (wallets non financés), aucune tokenisation/certificat/Move/DID, réconciliation unitaire seulement.

7. Work Already Existing in Java Backend

Présenté sans dévaloriser le travail existant — en distinguant systématiquement fonction métier vs fonction blockchain IOTA.

Le Backend Java possède déjà une plateforme d'échange énergétique complète et fonctionnelle, développée indépendamment du service IOTA :

Élément	Fonction métier (réelle)	Fonction blockchain IOTA (réelle)
Marketplace / matching	Agents producteurs/consommateurs, enchères, appariement — réel, fonctionnel	Aucune
Pricing	Références de prix, règles de règlement (<code>SettlementRule</code>) — réel	Aucune

Élément	Fonction métier (réelle)	Fonction blockchain IOTA (réelle)
Wallets	Comptes euros internes, mouvements atomiques, verrous anti-interblocage — réel	<code>iotaAddress</code> déclarative, jamais vérifiée ; balance IOTA toujours nulle, jamais lue depuis un nœud
Settlement	Règlement des transactions d'énergie, statuts PENDING/COMPLETED/FAILED — réel	Champ <code>iota_tx_id</code> présent en base mais jamais peuplé par une vraie transaction réseau
Token métier	« Jetons kWh vert » — lignes PostgreSQL avec cycle de vie (créé/transféré/consommé/expiré) — réel	Aucun token on-chain
Certificats	Saga complète réservation/vente/expiration, garantie d'origine — réel	<code>iotaTransactionHash</code> présent en base mais jamais alimenté par IOTA réel
Ledger / registre	Chaîne de hachés SHA-256 signée Ed25519, vérifiable, append-only — réel	Ancrage distant IOTA codé mais désactivé par défaut, jamais exercé
Outbox	Pattern outbox transactionnel générique, idempotent, utilisé par 6+ services — réel	Sert aussi de « puits » pour le registre local, pas pour IOTA
Sécurité	Identité Ed25519 propagée (passerelle), secret partagé inter-services, Keycloak/OIDC — réel	Sans lien avec IOTA
Réconciliation	3 sagas réconciliées quotidiennement (règlements, certificats, ancrages), alertent sans corriger — réel	La réconciliation d'ancrage ne trouve jamais d'écart puisque le mode IOTA n'est jamais actif
Digital Twin	Estimation SoC (EKF + Thévenin 2RC), décisions charge/décharge, validation par moteur de règles séparé — réel, plus avancé que la référence Python fournie par le client	Sans lien direct avec IOTA (passe éventuellement par le registre local, voir Section 20)
PostgreSQL	129+ tables (monolithe) + tables dédiées par microservice extrait — réel	Colonnes <code>iota.*</code> présentes mais structurellement jamais alimentées par un vrai réseau

Le travail Backend Java n'est en rien à refaire : c'est une plateforme métier mature. Ce qui manque n'est pas la logique métier, mais **la brique d'exécution réelle sur le réseau IOTA** — exactement ce que `iota-service` fournit déjà séparément.

8. Java Local Ledger vs Real IOTA

Ledger Java local

- Fichiers** : `usecase/Ledger.java` (contrat), `usecase/LedgerService.java` (implémentation), `domain/LedgerEntry.java` (entité), `domain/Canonical.java` (hachage), `infrastructure/registre/SignataireEd25519.java` (signature).
- Mécanisme** : chaque entrée contient `seq` (séquence croissante), `prevHash` (haché de l'entrée précédente), `payloadHash` = SHA-256(JSON canonique de la charge), `entryHash` = SHA-256(seq+prev+kind+entityType+entityId+payloadHash+timestamp), une **signature Ed25519 générique** (implémentation JDK pure, `domain/Ed25519Math.java`, **pas** le SDK IOTA), et un champ `network` valant `LOCAL`, `IOTA_PENDING` ou `IOTA`.
- Stockage** : table PostgreSQL `ledger_entries`, verrouillée en écriture par `pg_advisory_xact_lock` — une seule chaîne, cohérente même avec plusieurs instances du service.
- tx_id** : en mode local, c'est le `entryHash` lui-même — **une valeur calculée localement, jamais une référence réseau**.

Blockchain IOTA Rebased réelle (telle qu'utilisée par `iota-service`)

- Fichiers** : `iota-service/src/iota/client.ts`, `transaction/transactionService.ts`.
- Mécanisme** : un `Transaction` (PTB — Programmable Transaction Block) est construit via le SDK, signé avec un `Ed25519Keypair` du SDK IOTA, diffusé au réseau via `signAndExecuteTransaction`. Le réseau retourne un `digest` (identifiant de transaction reconnu par le réseau IOTA lui-même) et un statut d'exécution (`effects.status`) déterminé par le réseau, pas par le service appelant.
- Vérifiabilité par un tiers** : n'importe qui disposant du `digest` peut interroger n'importe quel nœud IOTA Devnet et retrouver la même transaction — **sans avoir à faire confiance à H2S**.

Pourquoi ce ne sont PAS la même chose

Propriété	Ledger Java local	IOTA Rebased réel
Où est stockée la preuve	PostgreSQL H2S uniquement	Réseau IOTA distribué
Qui peut vérifier sans faire confiance à H2S	Personne — il faut faire confiance à la base H2S et à sa clé de signature	N'importe quel nœud IOTA
Signature	Ed25519 générique, clé gérée par H2S (<code>H2S_LEDGER_SIGNING_KEY</code> , ou clé éphémère si absente)	Ed25519 du SDK IOTA, clé du wallet source
Identifiant	<code>entryHash</code> (SHA-256 local)	<code>digest</code> (identifiant réseau IOTA)
Objet on-chain	Aucun	Coin/objet réellement transféré (PTB)
État actuel dans le code Java	<code>network = LOCAL</code> pour 100% des entrées en configuration par défaut	N/A côté Java — c'est <code>iota-service</code> qui parle réellement au réseau

Conclusion, écrite explicitement comme demandé : le registre Java actuel n'est pas équivalent à une transaction IOTA Rebased. Le code Java (`LedgerService.java:94-103`) le documente lui-même : l'ancrage distant, quand il serait activé, ne ferait qu'ajouter une référence de transaction externe (`tx_id`) à une entrée déjà écrite et signée localement — **le registre local resterait, dans tous les cas, la structure de données de référence côté Java**, IOTA n'y ajoutant qu'une preuve externe optionnelle et aujourd'hui inactive.

9. Detailed Comparison

Fonction	Backend Java / <code>iota-connector</code>	IOTA Service TypeScript	État	Responsable futur (proposition)	Commentaire
Connexion IOTA	<code>AncrageIotaHttp</code> , HTTP JDK vers un nœud hypothétique	<code>IotaClient</code> du SDK officiel, Devnet	[LOCAL JAVA] / [EXISTANT TS]	iota-service	Java désactivé par défaut, jamais testé contre un nœud réel
Réseau	<code>h2s.ledger.mode</code> local/iota, pas de notion devnet/testnet/mainnet	<code>IOTA_NETWORK</code> , garde-fou anti-mainnet en code	[PARTIEL JAVA] / [EXISTANT TS]	iota-service	—
Wallet	Compte EUR interne + adresse IOTA déclarative	Keypair Ed25519 réel, adresse dérivée	[LOCAL JAVA] / [EXISTANT TS]	iota-service (custody) / Java (identité métier)	À séparer : identité métier (Java) vs clé cryptographique (TS)
Adresse	Chaîne déclarée par l'utilisateur ou dérivée du hash local	Dérivée cryptographiquement du keypair	[LOCAL JAVA] / [EXISTANT TS]	iota-service	—
Secret/keypair	Aucun secret IOTA manipulé	<code>Ed25519Keypair</code> , fichier JSON local (dev-only)	[NON IMPLÉMENTÉ JAVA] / [PARTIEL TS]	iota-service	Le wallet store TS doit évoluer avant production
Balance	<code>balance_iota</code> toujours 0 (jamais lu)	Lecture réseau réelle à chaque appel	[NON IMPLÉMENTÉ JAVA] / [EXISTANT TS]	iota-service	—
Transaction	Mouvement de compte EUR + entrée registre local	PTB réelle signée/exécutée sur Devnet	[LOCAL JAVA] / [EXISTANT TS]	iota-service	—
PTB	Absent	<code>Transaction</code> , <code>splitCoins</code> , <code>transferObjects</code>	[NON IMPLÉMENTÉ JAVA] / [EXISTANT TS]	iota-service	—
Digest	Champ candidat jamais peuplé (<code>AncrageIotaHttp.java:116</code>)	<code>response.digest</code> réel, retourné et persisté	[NON IMPLÉMENTÉ JAVA] / [EXISTANT TS]	iota-service génère, Java persiste la référence	—
ObjectId	Absent (colonnes DB confirmées absentes)	Absent également dans <code>iota-service</code> aujourd'hui (transferts de coin natif seulement)	[NON TROUVÉ] des deux côtés	À définir	Nécessaire seulement si assets/certificats on-chain sont visés
RequestId	<code>X-Request-Id</code> généralisé (<code>RequestIds</code> , <code>h2s-noyau</code>)	<code>requestId</code> obligatoire, motif <code>[A-Za-z0-9_-]{1,128}</code>	[EXISTANT JAVA] / [EXISTANT TS]	Java génère, propage tel quel	Convention à harmoniser — Section 15
Idempotence	3 couches (X-Request-Id, outbox+clé métier, contraintes DB uniques) — asymétrique sur le chemin synchrone historique	<code>INSERT...ON CONFLICT DO NOTHING</code> , atomique, testé sous concurrence	[EXISTANT JAVA] / [EXISTANT TS]	Chacun la sienne, cohérentes par construction	—
PostgreSQL	13 tables brique 8 + outbox	1 table (<code>iota_transactions</code>)	[EXISTANT] des deux côtés	Chacun sa base (à valider, Section 25)	—
Reconciliation	3 sagas quotidiennes, alertent sans corriger	Route dédiée, lecture seule, par requestId	[EXISTANT] des deux côtés	À faire correspondre	Java pourrait consommer <code>/api/iota/reconcile</code>
Retry	Rejeu d'ancrage distant toutes les 5 min (inopérant, IOTA désactivé)	Retry côté appelant recommandé, même requestId (documenté)	[PARTIEL JAVA] / [EXISTANT TS, côté appelant]	Backend Java à implémenter le retry HTTP	—
REST	31 endpoints publics + routes internes	9 endpoints documentés	[EXISTANT] des deux côtés	—	—
Sécurité	Secret partagé + identité Ed25519 propagée	Aucune authentification entrante	[EXISTANT JAVA] / [NON IMPLÉMENTÉ TS]	À concevoir ensemble	Section 16
Tokenization	Jetons kWh — lignes DB, pas on-chain	Absent	[LOCAL JAVA] / [NON IMPLÉMENTÉ TS]	À décider	Aucun des deux ne fait de tokenisation réelle on-chain aujourd'hui

Fonction	Backend Java / <i>iota-connector</i>	IOTA Service TypeScript	État	Responsable futur (proposition)	Commentaire
Green kWh	Concept métier réel (<i>EnergyToken</i>)	Absent du périmètre actuel	[EXISTANT JAVA – métier] / [NON IMPLÉMENTÉ TS]	Java conserve le métier	—
Certificate	Saga complète, réelle	Absent	[EXISTANT JAVA – métier] / [NON IMPLÉMENTÉ TS]	Java conserve le métier	—
Settlement	Réel, complet	Absent (hors périmètre)	[EXISTANT JAVA] / [NON IMPLÉMENTÉ TS]	Java	—
Move	Absent	Absent	[NON IMPLÉMENTÉ] des deux côtés	À étudier	Aucune preuve de smart contract Move nulle part
DID	<i>did:key</i> : local fabriqué ailleurs (monolithe), non résoluble	Absent	[PARTIEL JAVA – non conforme IOTA Identity] / [NON IMPLÉMENTÉ TS]	À décider	—
W3C Verifiable Credentials	Absent	Absent	[NON IMPLÉMENTÉ] des deux côtés	À décider	—
IOTA Streams	Remplacé par AES-256-GCM maison (<i>StreamService</i>), explicitement un substitut	Absent	[PARTIEL JAVA – substitut assumé] / [NON IMPLÉMENTÉ TS]	À décider	—

10. Responsibilities

Proposition — pas une vérité, à valider avec l'équipe.

Backend Java — candidats [PROPOSITION]

- Utilisateurs, sites, énergie (production/consommation) — [CONFIRMÉ PAR LE CODE] déjà porté par Java
- Digital Twin (SoC, décisions batterie) — [CONFIRMÉ PAR LE CODE]
- Marketplace, matching, pricing — [CONFIRMÉ PAR LE CODE]
- Settlement métier, certificats métier — [CONFIRMÉ PAR LE CODE]
- Décisions métier, orchestration — [CONFIRMÉ PAR LE CODE]
- PostgreSQL métier (13 tables brique 8 + reste du monorepo) — [CONFIRMÉ PAR LE CODE]
- Génération/portage du *requestId* métier — [PROPOSITION], Java a déjà l'infrastructure (*RequestIds*)

IOTA Service — candidats [PROPOSITION]

- Réseau IOTA, SDK *@iota/iota-sdk* — [CONFIRMÉ PAR LE CODE] déjà porté par TS
- Wallets/keypairs, adresses — [CONFIRMÉ PAR LE CODE]
- Balances réelles — [CONFIRMÉ PAR LE CODE]
- PTB, transactions on-chain — [CONFIRMÉ PAR LE CODE]
- Digest, (futur) objectId — [CONFIRMÉ PAR LE CODE] pour digest ; objectId [NON IMPLÉMENTÉ]
- Preuves IOTA (au sens réseau) — [CONFIRMÉ PAR LE CODE]
- Réconciliation blockchain — [CONFIRMÉ PAR LE CODE]
- Erreurs IOTA (traduction des erreurs SDK en statuts HTTP propres) — [CONFIRMÉ PAR LE CODE]

Zone à trancher explicitement [À VALIDER] : le registre local Java (*Ledger*) reste-t-il un composant Java (audit interne), ou une partie de sa responsabilité (preuve externe vérifiable) migre-t-elle vers *iota-service* ? Voir Section 12 (options, sans choix imposé).

11. Integration Readiness

"Readiness for Backend Java Integration"

Question posée : si le responsable Backend Java veut commencer aujourd'hui à appeler le service IOTA, qu'est-ce qui est prêt et qu'est-ce qui manque ?

Élément	État actuel	Preuve dans le code	Manque	Priorité
Service démarrable	[IMPLÉMENTÉ]	src/index.ts , npm run dev / start	Rien de bloquant	—
/health	[IMPLÉMENTÉ, TESTÉ]	server.ts:80-83 , tests/health.test.ts	Rien	—
/api/iota/network	[IMPLÉMENTÉ, TESTÉ]	walletController.getNetwork()	Rien	—
Wallet (créer/lire/lister)	[IMPLÉMENTÉ, TESTÉ]	walletController.ts , tests/iota-wallet-api.test.ts	Store fichier dev-only	Haute avant prod
Balance	[IMPLÉMENTÉ, TESTÉ]	tests/iota-balance.test.ts	Rien pour Devnet	—
Transaction (créer)	[IMPLÉMENTÉ, TESTÉ]	tests/iota-transaction-devnet.test.ts	Fonds réels non démontrés	Moyenne
Récupération transaction (par digest)	[IMPLÉMENTÉ, TESTÉ]	transactionController.getTransaction	Rien	—
RequestId	[IMPLÉMENTÉ, TESTÉ]	validation regex, tests dédiés	Convention Java→TS à harmoniser	Haute
Idempotence	[IMPLÉMENTÉ, TESTÉ]	postgresTransactionRepository.ts , tests/iota-transaction-repository-postgres.test.ts	Rien	—
Persistence	[IMPLÉMENTÉ]	1 table, migration forward-only	Wallets pas en DB	Moyenne
Digest	[IMPLÉMENTÉ]	response.digest	objectId absent (non nécessaire au périmètre actuel)	Basse
Statuts	[IMPLÉMENTÉ, TESTÉ]	pending/confirmed/failed , codes HTTP nuancés	Rien	—
Erreurs	[IMPLÉMENTÉ, TESTÉ]	format uniforme {error:{code,message}}	Rien	—
Retry	[DOCUMENTÉ, côté appelant]	docs/integration-backend.md §12	Implémentation cliente à faire côté Java	Haute
Réconciliation	[IMPLÉMENTÉ, TESTÉ] (unitaire)	tests/iota-reconciliation-unit.test.ts	Réconciliation par lot absente	Basse
Authentication/Authorization	[NON IMPLÉMENTÉ]	Confirmé absent, documenté comme tel	Mécanisme entier à définir	Critique avant tout déploiement partagé
Documentation	[EXISTANT]	docs/integration-backend.md , complet et testé en cohérence avec le code	Rien de bloquant	—
OpenAPI/Swagger	[NON TROUVÉ]	Aucun fichier OpenAPI généré côté iota-service	À produire si l'équipe le souhaite	Basse (le doc Markdown suffit aujourd'hui)
Tests	[IMPLÉMENTÉ]	11 fichiers, y compris contractuel	Rien	—
Docker/containerisation	[NON TROUVÉ]	Aucun Dockerfile dans iota-service (contrairement à iota-connector qui en a un)	À créer si déploiement conteneurisé visé	Moyenne
Configuration environnement	[IMPLÉMENTÉ]	src/config/index.ts , variables d'env documentées	URL de prod/port réel [À VALIDER]	Haute

Verdict global : le service est **prêt côté contrat REST et comportement fonctionnel** pour un premier appel de test (Devnet, sans authentification). Il n'est **pas prêt pour un déploiement partagé avec le Backend** tant que l'authentification et la stratégie de wallet store ne sont pas décidées.

12. First Java → IOTA Integration Milestone

"First Integration Milestone"

Documentation uniquement — rien de ce qui suit n'est implémenté.

```

Backend Java
|
| POST /api/iota/transactions (REST/JSON)
| requestId : <convention à valider, Section 15>
|
▼
IOTA Service (transactionController.createTransaction)
|
| validation (regex requestId/walletId/toAddress/amount)
| idempotence (tryCreatePending, PostgreSQL)
| construction PTB (Transaction, splitCoins, transferObjects)
| signature (Ed25519Keypair du wallet source)
| exécution (signAndExecuteTransaction)
|
▼
IOTA Rebased – Devnet
|
| digest / effects.status
|
▼
IOTA Service (markConfirmed / markFailed)
|
| response REST/JSON {requestId, status, digest?, ...}
|
▼
Backend Java (persistance de la référence, à définir où – Section 17)

```

Étape	Classe Java (existante ou à créer)	Classe TS (existante)	Endpoint	Statut
Backend décide d'ancrer/transférer	<code>usecase/AncrageEntrant</code> ou nouveau usecase dédié [À CRÉER – proposition]	—	—	[PROPOSITION]
Backend génère/porte requestId	<code>com.h2s.noyau.correlation.RequestIds</code> [EXISTANT, réutilisable]	—	—	[EXISTANT côté Java]
Backend appelle iota-service	Nouveau client HTTP [À CRÉER – proposition, sur le modèle de ClientInterne]	<code>server.ts</code> (routage)	POST <code>/api/iota/transactions</code>	[NON IMPLÉMENTÉ]
iota-service valide + idempotence	—	<code>transactionController.createTransaction</code> , <code>transactionService.sendTransaction</code>	—	[EXISTANT]
iota-service exécute sur IOTA	—	<code>transactionService.sendTransaction</code> (PTB)	—	[EXISTANT]
iota-service répond	—	<code>transactionController.createTransaction</code>	réponse {status, digest, ...}	[EXISTANT]
Backend persiste la référence	Nouveau champ/table [À DÉFINIR – Section 17]	—	—	[NON IMPLÉMENTÉ]

Tests nécessaires avant ce premier appel réel : test d'intégration Java→TS en environnement de développement partagé, avec un wallet Devnet créé via `POST /api/iota/wallets`, et vérification que le `requestId` Java (`RequestIds.current()`) est accepté par la regex `^[A-Za-z0-9_-]{1,128}$` du service TS [À VÉRIFIER – compatibilité de format] .

13. Target Architecture



Ce schéma est la **cible discutée avec l'équipe**, [PROPOSITION – À VALIDER]. Il n'est pas contredit par le code existant (les deux services sont architecturalement compatibles avec ce schéma : `iota-connector` a déjà un port `AncrageDistant` isolant l'ancrage externe, `iota-service` a déjà un contrat REST/JSON stable), mais **aucune ligne de code ne le réalise aujourd'hui**.

Point à trancher explicitement [À VALIDER] : `iota-connector` Java devient-il l'appelant direct de `iota-service`, ou un autre composant Java (le monolithe, ou un futur point d'entrée dédié) ? Le port `AncrageDistant` (`usecase/depot/AncrageDistant.java`) est le point d'extension naturel si `iota-connector` reste l'appelant — c'est un fait d'architecture existant, pas une recommandation imposée.

14. REST Contract

API réellement existante [EXISTANT – vérifié dans le code, testé]

Contrat intégral déjà documenté par `iota-service/docs/integration-backend.md` (lu intégralement pour ce document) :

Méthode	URL	Request	Response (succès)	Erreurs	Idempotence	Testé
GET	<code>/health</code>	—	<code>{status, service}</code>	—	N/A	Oui
GET	<code>/api/iota/network</code>	—	<code>{network}</code>	—	N/A	Oui
GET	<code>/api/iota/wallets</code>	—	<code>{wallets: [...]}</code>	—	N/A	Oui
POST	<code>/api/iota/wallets</code>	<code>{walletId}</code>	201 <code>{walletId, address, network, createdAt}</code>	400, 409	Non (par design — création)	Oui
GET	<code>/api/iota/wallets/:walletId</code>	—	200 <code>wallet</code>	400, 404, 500	N/A	Oui
GET	<code>/api/iota/wallets/:walletId/balance</code>	—	200 <code>{walletId, address, network, balance}</code>	404, 502	N/A	Oui
POST	<code>/api/iota/transactions</code>	<code>{requestId, fromWalletId, toAddress, amount}</code>	200/201/202 selon statut réel	400×5, 403, 404	Oui, obligatoire	Oui
GET	<code>/api/iota/transactions/:digest</code>	—	200 <code>{digest, status, error?, timestampMs?}</code>	404, 502	N/A	Oui
POST	<code>/api/iota/reconcile</code>	<code>{requestId}</code>	200 <code>{requestId, postgresStatus, iotaStatus?, outcome, checkedAt}</code>	400, 404	N/A (lecture seule)	Oui

Classement demandé (Partie 3 du prompt) :

- `/health`, `/api/iota/network`, `/api/iota/wallets` (GET/POST), `/api/iota/wallets/:id`, `/api/iota/wallets/:id/balance`, `/api/iota/transactions` (POST/GET par digest), `/api/iota/reconcile` : [EXISTANT ET TESTÉ], tous les 9 endpoints du contrat indicatif fourni par l'utilisateur **existent réellement** dans le code — vérifié, pas supposé.

API proposée pour l'intégration Java [PROPOSITION – À VALIDER]

Aucun endpoint supplémentaire n'est strictement nécessaire pour un premier raccordement — le contrat existant suffit. Deux ajouts **pourraient** être discutés, sans être décidés ici :

- `POST /api/iota/auth` ou en-tête d'authentification à définir — Section 16.
- `POST /api/iota/reconcile/batch` si le Backend a besoin de réconcilier plusieurs `requestId` en une fois (aujourd'hui non implémenté côté TS).

15. RequestId and Idempotence

Question	Réponse vérifiée
Où <code>requestId</code> existe côté Java	<code>com.h2s.noyau.correlation.RequestIds</code> (<code>libs/h2s-noyau</code>), en-tête <code>X-Request-Id</code> , MDC de log — <code>[EXISTANT]</code>
Qui le génère côté Java	<code>RequestIds.accept()</code> (lit l'en-tête entrant) ou <code>RequestIds.generate()</code> (UUID si absent) — <code>[EXISTANT]</code>
Qui le transmet côté Java	<code>ClientInterne</code> (appels au monolithe), propagation systématique — <code>[EXISTANT]</code>
Où <code>requestId</code> existe côté TS	Champ obligatoire de <code>POST /api/iota/transactions</code> , motif <code>^[A-Za-z0-9_-]{1,128}\$</code> — <code>[EXISTANT]</code>
Qui le génère côté TS	Le Backend appelant — le service ne génère jamais de <code>requestId</code> lui-même, par design (<code>docs/integration-backend.md</code> §7) — <code>[EXISTANT, confirmé]</code>
Qui le persiste	Java : MDC/logs seulement (pas de table dédiée au <code>X-Request-Id</code> de tracing) ; TS : colonne <code>request_id</code> <code>UNIQUE</code> de <code>iota_transactions</code> — <code>[EXISTANT]</code> des deux côtés, formes différentes
Détection de doublons	Java (outbox) : <code>JournalRecus</code> , PK composite (<code>consommateur</code> , <code>cle</code>) ; TS : contrainte PostgreSQL <code>UNIQUE(request_id)</code> + <code>INSERT...ON CONFLICT</code> — <code>[EXISTANT]</code> des deux côtés
Comportement d'un retry	TS : même <code>requestId</code> → même résultat retourné, aucune ré-exécution — règle documentée et imposée à l'appelant (<code>docs/integration-backend.md</code> §12)
Comportement d'une requête identique	Idem — testé (<code>tests/contract.test.ts</code> , cas « Rejouer le même requestId »)
Différence Java / IOTA Service	Le <code>X-Request-Id</code> Java est un identifiant de corrélacion/tracage (régénéré librement à chaque appel HTTP interne) ; le <code>requestId</code> du service IOTA est une clé métier d'idempotence d'opération (doit rester identique à travers tous les retries d'une même opération logique). Ce sont deux notions différentes qui portent le même nom — point d'attention explicite pour l'intégration.
Contraintes PostgreSQL	Java : <code>idempotency_key</code> unique sur <code>settlement_engine</code> , <code>financial_transactions</code> , <code>payments</code> , etc. ; TS : <code>request_id</code> unique sur <code>iota_transactions</code> — mêmes principes, implémentations indépendantes
Risque de double transaction IOTA	Si le Backend Java générerait un nouveau <code>requestId</code> à chaque retry (au lieu de réutiliser celui de la tentative logique), l'idempotence du service TS serait contournée et une double transaction IOTA deviendrait possible — risque réel si la convention n'est pas respectée , voir Section 27

Convention proposée `[À VALIDER]` : réutiliser le `X-Request-Id` / `RequestIds.current()` Java déjà généré pour l'opération métier d'origine comme `requestId` du corps JSON envoyé à `iota-service` , en s'assurant qu'il respecte le motif `^[A-Za-z0-9_-]{1,128}$` (à vérifier : les UUID Java standard avec tirets sont compatibles) `[À VÉRIFIER]` .

16. Security

Java

- `X-H2S-Cle-Interne` (secret partagé, comparaison temps constant, échec fermé) — `[EXISTANT]`
- Identité Ed25519 signée par la passerelle, retransmise (jamais re-signée) — `[EXISTANT]`
- Keycloak/OIDC pour les utilisateurs finaux (`@Authenticated` , `@RolesAllowed`) — `[EXISTANT]`
- `X-Request-Id` propagé systématiquement — `[EXISTANT]`
- Passerelle interne (`gateway/`) comme point de résolution d'identité unique — `[EXISTANT]`

IOTA Service — mécanisme actuel réellement trouvé

- **Aucun.** Le code et sa documentation le confirment explicitement (`docs/integration-backend.md` §13 : « Authentification Backend ↔ IOTA Service : `[À VALIDER]` — non implémentée »). `[NON IMPLÉMENTÉ]`
- Emplacement prévu pour un futur middleware, déjà identifié dans le code : `handleRequest()` dans `src/server.ts` , avant le routage.

Ce qu'il faut prévoir pour `Backend Java → IOTA Service` `[À VALIDER avec l'équipe – aucune solution choisie ici]`

- **Authentication** : options possibles à évaluer — secret partagé (même patron que `GardeInterne` Java, le plus cohérent avec l'existant), API key statique, ou OAuth2 client-credentials via Keycloak si le service IOTA rejoint le périmètre géré par la passerelle.
- **Authorization** : aucune notion de rôle n'est nécessaire côté `iota-service` si seul le Backend l'appelle (pas d'utilisateur final direct) — à confirmer.
- **Secret/API key** : si retenu, doit suivre la même discipline que Java (`h2s.interne.cle`) — jamais en dur, variable d'environnement obligatoire.
- **mTLS** : envisageable si le déploiement réseau le justifie — non nécessaire pour un premier jalon de développement.
- **OAuth2/client credentials** : pertinent seulement si `iota-service` rejoint l'écosystème Keycloak — sujet d'équipe, pas une décision technique isolée.
- **Vault** : pertinent pour le remplacement du wallet store fichier de `iota-service` (Section 23/24) — distinct de l'authentification Backend↔Service.
- **Gestion des keypairs** : aujourd'hui fichier JSON en clair côté TS (dev-only, assumé) ; Java ne gère aucun keypair IOTA.
- **Logs/secrets** : les deux projets respectent déjà la règle « jamais de secret en dur » — Java via variables d'environnement sans défaut en production, TS via `.env` /variables d'environnement documentées.
- **PostgreSQL** : bases séparées aujourd'hui par défaut (`iota-service` vs `h2s`) — topologie de production [À VALIDER] .

17. Data Persistence

Donnée	Backend Java	PostgreSQL	IOTA	Commentaire
Raw telemetry (mesures batterie)	<code>digital-twin</code>	Oui (<code>storages</code> , EKF/Thévenin)	Non	Off-chain par nature — haute fréquence
Production / consommation	<code>dashboard</code> / <code>enedis</code>	Oui	Non	Off-chain
Digital Twin (état, décisions)	<code>digital-twin</code>	Oui	Non directement (peut passer par le registre, voir Section 20)	—
Predictions	<code>prediction</code> (LightGBM via <code>ai-engine</code>)	Oui	Non	Off-chain
Prices	<code>iota-connector</code> (marché)	Oui	Non	Off-chain
Matching	<code>iota-connector</code> (<code>MarketService</code>)	Oui	Non	Off-chain
Settlement	<code>iota-connector</code> (<code>SettlementService</code>)	Oui	Non aujourd'hui (<code>iota_tx_id</code> jamais peuplé)	Candidat à preuve externe si décidé
Wallet identity (métier)	<code>iota-connector</code> (<code>Wallet</code> , EUR)	Oui	Non	Compte EUR, pas un wallet crypto
Wallet identity (cryptographique)	Aucun	Non (fichier local TS)	Adresse dérivée du keypair, publique	<code>iota-service</code> uniquement
Address	<code>iota-connector</code> (déclarative) / <code>iota-service</code> (dérivée)	Oui (Java) / Non (TS, fichier)	Oui (publique, dérivée cryptographiquement)	Deux notions différentes, à ne pas confondre
Transaction digest	Absent côté Java (jamais peuplé)	Oui côté TS (<code>iota_transactions.digest</code>)	Oui (réseau)	À faire remonter côté Java si intégration réalisée
ObjectId	Absent des deux côtés	—	—	Non nécessaire au périmètre actuel (transferts de coin natif)
Token (kWh vert)	<code>iota-connector</code> (<code>EnergyToken</code>)	Oui	Non	Métier pur, pas on-chain
Certificate	<code>iota-connector</code> (<code>GreenCertificate</code>)	Oui	Non (<code>iotaTransactionHash</code> jamais peuplé)	Candidat à ancrage si décidé
Proof (registre local)	<code>iota-connector</code> (<code>LedgerEntry</code>)	Oui	Non par défaut (<code>network=LOCAL</code>)	Voir Section 8
Audit	<code>iota-connector</code> (<code>AuditIota</code> , outbox)	Oui	Non	Off-chain

Principe évoqué dans la demande, à noter explicitement comme non validé par l'équipe [PROPOSITION — non confirmée dans les documents de l'équipe] : « *Only put on-chain what a third party needs to verify without trusting H2S. Everything else stays in PostgreSQL.* » Ce principe est cohérent avec l'architecture observée (le registre local Java garde déjà tout en PostgreSQL par défaut, IOTA n'intervenant qu'en complément optionnel), mais il n'a pas été retrouvé formulé ainsi dans la documentation de l'équipe — à faire valider explicitement s'il doit devenir un principe directeur.

18. On-chain / Off-chain

(Matrice fusionnée avec la Section 17 pour éviter la duplication — voir tableau ci-dessus, qui couvre exactement les colonnes demandées.)

19. Reconciliation

	Backend Java	IOTA Service
Existe	Oui — <code>Reconciliations</code> (3 sagas : règlements, certificats, ancrages)	Oui — <code>reconcileTransaction</code>
Fréquence	Quotidienne (cron <code>0 20 3 * * ?</code>)	À la demande (<code>POST /api/iota/reconcile</code>)
Portée	Toute la journée précédente	Une transaction (<code>requestId</code>) à la fois
Comportement	Alerte, ne corrige jamais (choix explicite et documenté)	Lecture seule, ne modifie jamais PostgreSQL (choix explicite et documenté)
Limite connue	Sans nœud IOTA actif, la réconciliation d'ancrage ne trouve jamais d'écart (rien à comparer)	Un <code>pending</code> sans digest n'est pas réconciliable (limite documentée dans le code)

Cohérence entre les deux systèmes de réconciliation `[À VALIDER]` : si l'intégration se réalise, la réconciliation Java devra probablement appeler `POST /api/iota/reconcile` pour chaque référence IOTA qu'elle détient, plutôt que réimplémenter une logique de comparaison — à discuter, pas décidé ici.

20. Digital Twin Integration

`[EXISTANT – confirmé par le code, pour la partie Digital Twin ; PARTIEL pour le lien vers IOTA]`

- `digital-twin` (`services/digital-twin`) écrit ses décisions (`BatteryDecision`) et émet des événements outbox (`AnnoncesDecisionOutbox` , sujets `h2s/evt/decision/validee` , `h2s/evt/decision/appliquee` , `h2s/evt/batterie/veille-imposee`) dans **sa propre** transaction.
- Le mécanisme générique d'ancrage via outbox (`AncrageEntrant` , `EvenementsInternesResource` dans `iota-connector`) **existe et fonctionne**, indépendamment de `digital-twin` .
- Ce qui n'a pas été confirmé ligne à ligne** : l'émission effective, par `digital-twin` , du sujet spécifique `h2s/evt/registre/ancrer` pour chacune de ses décisions. L'audit dédié précédent a confirmé que `digital-twin` émet des événements outbox liés aux décisions, et que `iota-connector` sait consommer un ordre d'ancrage générique — mais le câblage exact entre les deux, pour ce flux précis, mériterait une relecture ciblée de `digital-twin/infrastructure/evenements/AnnoncesDecisionOutbox.java` si une confirmation à 100% est requise. `[À VÉRIFIER]`
- Un événement Digital Twin n'est, dans tous les cas, jamais envoyé vers `iota-service` aujourd'hui** — seul le registre local Java (`Ledger`) serait, au mieux, concerné, jamais le réseau IOTA réel. **Ceci n'est pas une supposition : c'est l'absence confirmée de toute référence à `iota-service` dans tout `iota-connector` et `digital-twin` .**

21. Energy Event Integration

```
Production
↓
Digital Twin / validation          [EXISTANT – digital-twin]
↓
Backend Java (decision, settlement) [EXISTANT – iota-connector]
↓
IOTA Service                      [NON IMPLÉMENTÉ – aucun appel existant]
↓
IOTA Rebased                      [EXISTANT côté iota-service seul, jamais atteint depuis Java]
```

Aucun format JSON d'événement énergétique destiné à IOTA n'a été trouvé dans le code Java ni dans `iota-service` . Le format ci-dessous est une **proposition conceptuelle**, non un contrat existant :

```
// [PROPOSITION] – non présent dans le code, à valider avec l'équipe
{
  "requestId": "h2s-decision-<id>",
  "eventType": "ENERGY_SETTLEMENT",
  "siteId": 123,
  "amountKwh": "45.230",
  "payloadHash": "<sha256 déjà calculé côté Java par Canonical.hash(>)",
  "sourceType": "SOLAR"
}
```

`[PROPOSITION – ne pas présenter comme un contrat officiel]`

22. Tokenization / Certificates / Move

Élément	Logique métier Java	Stockage PostgreSQL	Vraie opération IOTA	Smart contract	SDK	Test	Statut réel
Green kWh / <code>EnergyToken</code>	Oui, réel (<code>TokenService</code>)	Oui (<code>energy_tokens</code>)	Non	Non	Non	Tests Java (<code>TokenTraceStreamTest</code>)	[EXISTANT métier – NON on-chain]
<code>Settlement</code>	Oui, réel	Oui	Non (<code>iota_tx_id</code> jamais peuplé)	Non	Non	Oui	[EXISTANT métier – NON on-chain]
<code>GreenCertificate</code> / <code>GuaranteeOfOrigin</code>	Oui, réel (saga complète)	Oui	Non (<code>iotaTransactionHash</code> jamais peuplé)	Non	Non	Oui (<code>SagasTest</code>)	[EXISTANT métier – NON on-chain]
DID	<code>did:key:</code> fabriqué localement ailleurs dans le monolithe, non résoluble	Oui (référéncé)	Non (pas IOTA Identity)	Non	Non	[À VÉRIFIER]	[PARTIEL – non conforme au standard IOTA Identity]
W3C Verifiable Credentials	Absent	Absent	Absent	Absent	Absent	Absent	[NON IMPLÉMENTÉ]
Move	Absent des deux côtés	—	—	Absent	Absent	Absent	[NON IMPLÉMENTÉ]

23. Required Changes in Java

Analyse uniquement — aucune modification n'a été effectuée.

Modifications probablement nécessaires [PROPOSITION]

Fichier/Classe	Modification envisagée	Raison	Impact	Priorité
<code>usecase/depot/AncrageDistant.java</code> (interface existante)	Nouvelle implémentation <code>IotaServiceHttp</code> en remplacement/complément de <code>AncrageIotaHttp</code>	Isoler l'appel réel vers <code>iota-service</code> derrière le port déjà existant	Faible — le port est déjà découplé de <code>LedgerService</code>	Haute si l'Option C (Section 12 du prompt, cf. audit précédent) est retenue
<code>application.yml</code> (<code>h2s.ledger.iota.*</code>)	Ajouter des clés <code>h2s.iota-service.base-url</code> , <code>h2s.iota-service.cle</code> [PROPOSITION de nommage]	Configurer le nouveau client	Faible	Haute
Nouvelle classe cliente (nom à définir)	Créer, sur le modèle de <code>ClientInterne.java</code> , un client HTTP vers <code>iota-service</code>	Réutiliser le patron déjà validé (secret partagé, retry limité aux méthodes sûres, <code>X-Request-Id</code>)	Moyen	Haute
<code>LedgerEntry</code> / <code>GreenCertificate</code> / <code>FinancialTransaction</code>	Décider si <code>txId</code> / <code>iotaTransactionHash</code> doivent être alimentés par la réponse réelle de <code>iota-service</code>	Aujourd'hui ces champs existent mais ne sont jamais peuplés par un vrai réseau	Moyen	Moyenne — dépend de l'Option choisie en Section 12 (audit précédent)

Modifications possibles [PROPOSITION, non urgentes]

- Étendre `RequestIds` (déjà existant) pour documenter explicitement son usage comme `requestId` métier vers `iota-service`, si cette convention est validée (Section 15).
- Ajouter une entrée de configuration désactivant explicitement `AncrageIotaHttp` si l'Option C est retenue (éviter deux chemins actifs simultanément).

Modifications à NE PAS faire [explicite, pour éviter la duplication]

- Ne pas réimplémenter un SDK IOTA en Java.** Le pom.xml documente déjà pourquoi ce choix a été évité (risque d'écosystème non tranché) — `iota-service` remplit ce rôle.
- Ne pas dupliquer le mécanisme d'idempotence PostgreSQL de `iota-service` côté Java.** L'idempotence Java existante (`AncrageEntrant`, `WalletService`) reste valide pour le métier EUR/registre local ; elle n'a pas besoin de connaître l'idempotence interne de `iota-service`.

- **Ne pas modifier** `LedgerService` / `Ledger` pour l'instant — son rôle de registre interne reste valable indépendamment de la décision d'intégration (voir options Section 12 de l'audit précédent).

24. Required Changes in iota-service

Identification uniquement — rien n'a été créé.

Fonctionnalité	Statut
Authentification entrante (Backend → service)	[À IMPLÉMENTER] — emplacement déjà identifié (<code>server.ts</code> , avant routage)
Production event / energy proof	[À IMPLÉMENTER] si l'ancrage d'événements énergétiques réels est visé — absent aujourd'hui
Asset / certificate on-chain	[À IMPLÉMENTER] si décidé — absent, nécessiterait probablement un modèle objectId
Tokenization	[À IMPLÉMENTER] si décidé — absent
Move	[À IMPLÉMENTER] si décidé — absent, sujet non trivial
Settlement (au sens IOTA)	[À IMPLÉMENTER] si décidé
External event / callback	[NON TROUVÉ] — le service est aujourd'hui strictement synchrone requête/réponse, pas d'événements sortants
Proof verification	[PARTIEL] — la réconciliation vérifie un digest déjà connu ; pas de vérification de preuve tierce généralisée
Object lookup	[NON IMPLÉMENTÉ] — pas de notion d'objectId dans le périmètre actuel
Health / readiness	[EXISTANT] (<code>/health</code>) — pas de distinction liveness/readiness séparée
Audit (logs structurés)	[NON TROUVÉ] — logs <code>console.log</code> simples, pas de format structuré dédié observé
Callbacks/events sortants	[NON IMPLÉMENTÉ]

25. Git / Repository Strategy

[EXISTANT — état constaté ; options comparées, aucune décision]

- `java_backend_H2S` : repository GitLab d'équipe (`platform.git`), branche `dev` , nombreuses branches de fonctionnalités actives.
- `iota-service` : projet local. **Statut Git non vérifié dans le détail lors de cet audit** (aucune commande `git init` / `status` exécutée dans `iota-service` pour respecter la consigne de ne modifier aucun état Git) — [À VÉRIFIER] si un dépôt Git local existe déjà.

Options, comparées sans recommandation imposée

Option	Avantages	Inconvénients
1. Repository GitLab séparé	Indépendance de déploiement/CI-CD, ownership clair, cohérent avec l'architecture microservices déjà en place côté Java	Coordination de versions entre deux dépôts, deux pipelines à maintenir
2. Intégration dans le repository Java	Un seul endroit pour tout voir, CI/CD potentiellement unifiée	Casse l'indépendance de langage/déploiement déjà voulue par le principe « Backend décide, service IOTA exécute » ; le monorepo Java est déjà très large
3. Monorepo dédié (Java + IOTA service, séparé du reste)	Compromis entre les deux	Complexité de tooling (deux écosystèmes, Maven + npm)
4. Autre organisation	—	À définir par l'équipe

Rappel explicite de la contrainte du prompt : la décision appartient à l'équipe ; ce document ne recommande pas d'héberger le service sur un compte GitHub personnel comme dépôt de production sans validation explicite de l'équipe.

26. Migration Roadmap

Phase	Objectif	Fichiers concernés	Dépendances	Critère de validation	Statut actuel
0 — Validation architecture	Choisir entre Options A/B/C (Section 12, audit iota-connector)	—	Décision équipe	Compte-rendu écrit	[NON DÉMARRÉ]
1 — Contrat REST Java↔IOTA	Valider le contrat existant (<code>docs/integration-backend.md</code>) avec le responsable Backend	<code>iota-service/docs/integration-backend.md</code>	Phase 0	Accord explicite du responsable Backend	[PARTIEL — document prêt, validation en attente]
2 — Sécurité	Choisir et implémenter l'authentification Backend↔Service	<code>src/server.ts</code> (TS), nouveau client (Java)	Phase 1	Test d'appel authentifié réussi	[NON DÉMARRÉ]

Phase	Objectif	Fichiers concernés	Dépendances	Critère de validation	Statut actuel
3 — Premier appel Java → IOTA Service	Réaliser le jalon de la Section 12	Nouveau client Java, aucun changement TS nécessaire	Phase 2	Appel réel réussi en environnement de dev	[NON DÉMARRÉ]
4 — Ancrage réel d'un événement	Premier événement métier réellement transmis	À définir (Section 9 questions)	Phase 3	Digest réel obtenu et vérifiable	[NON DÉMARRÉ]
5 — Persistance des références IOTA	Stocker <code>digest</code> /statut côté Java	<code>LedgerEntry</code> /entité concernée	Phase 4	Champ peuplé en base, vérifié	[NON DÉMARRÉ]
6 — Réconciliation	Relier réconciliation Java ↔ <code>POST /api/iota/reconcile</code>	<code>Reconciliations.java</code> (Java)	Phase 5	Écart détecté et alerté correctement en test	[NON DÉMARRÉ]
7 — Production énergétique	Premier flux production→décision→ancrage réel	<code>digital-twin</code> , <code>iota-connector</code> , <code>iota-service</code>	Phase 6	À définir avec l'équipe	[NON DÉMARRÉ]
8 — Tokenization	Décision et implémentation éventuelle	—	Décision équipe	—	[NON DÉMARRÉ – décision non prise]
9 — Certificates	Décision et implémentation éventuelle on-chain	—	Décision équipe	—	[NON DÉMARRÉ – décision non prise]
10 — Move smart contracts	Étude de faisabilité	—	Décision équipe	—	[NON DÉMARRÉ]
11 — Testnet	Bascule Devnet→Testnet	<code>src/config/index.ts</code> (garde-fou déjà présent)	Phases précédentes validées	Tests de non-régression sur Testnet	[NON DÉMARRÉ]
12 — Production	Bascule vers Mainnet	Garde-fou explicite déjà bloquant (<code>config/index.ts:53-58</code>)	Toutes les phases précédentes, validation équipe complète	Décision explicite d'équipe, aucun raccourci technique possible (le code refuse Mainnet hors production)	[NON DÉMARRÉ]

Aucune phase n'est considérée comme terminée sans preuve — conformément à la consigne, ce tableau ne marque « terminé » aucune phase au-delà de ce que le code démontre déjà (Phase 1 partiellement prête grâce au contrat déjà rédigé).

27. Risks

Risque	Description	Impact	Probabilité (qualitative)	Mitigation	Décision nécessaire
Double source de vérité	Registre local Java + preuve IOTA réelle coexistent sans hiérarchie claire	Confusion sur quelle preuve fait foi en cas d'audit	Moyenne si l'intégration avance sans clarifier ce point	Documenter explicitement laquelle prévaut (Section 12, audit <code>iota-connector</code>)	[À VALIDER]
Double ledger	Le ledger Java et <code>iota_transactions</code> (TS) enregistrent chacun un état, potentiellement divergent	Incohérence lors d'un audit croisé	Moyenne	Réconciliation croisée explicite (Section 19)	[À VALIDER]
Double transaction IOTA	<code>requestId</code> mal réutilisé lors d'un retry Java	Transfert de valeur dupliqué sur IOTA réel	Faible si la convention Section 15 est respectée, sinon réelle	Convention stricte + tests d'intégration dédiés	[À VALIDER – convention]
Idempotence insuffisante côté chemin synchrone Java	<code>ancrerSynchrone</code> n'est délibérément pas idempotent	Entrée de registre local dupliquée (pas une transaction IOTA)	Faible (comportement connu et assumé, pas un bug)	Basculer les émetteurs restants vers l'outbox (déjà noté dans le code Java comme HP-54)	Décision de calendrier équipe
Perte de réponse (timeout réseau)	Un <code>POST /api/iota/transactions</code> peut timeout après diffusion réelle sur IOTA	Java pourrait croire l'opération échouée alors qu'elle a réussi on-chain	Moyenne en environnement réseau instable	Retry avec le même <code>requestId</code> (déjà la règle documentée côté TS)	Implémentation cliente Java à faire
Secrets / custody	Wallet store TS en fichier JSON clair, dev-only	Compromission si déployé tel quel en production	Élevée si déployé sans changement	Remplacer par un coffre-fort avant toute mise en production réelle	[À VALIDER – solution]
Dépendance réseau IOTA	Toute panne/latence du réseau Devnet/Testnet impacte directement les appels <code>iota-service</code>	Backend bloqué si appel synchrone sans timeout adapté	Moyenne	Timeout et stratégie de retry à définir côté Java (Section 11, <code>docs/integration-backend.md</code> §11 marque déjà [À VALIDER])	[À VALIDER]

Risque	Description	Impact	Probabilité (qualitative)	Mitigation	Décision nécessaire
Migration Devnet → Testnet → Mainnet	Bascule prématurée	Perte de fonds réels si mal préparée	Faible (garde-fou de code déjà bloquant)	Garde-fou déjà en place (<code>config/index.ts</code>), à ne jamais contourner	Validation équipe avant toute levée
Tokenisation / smart contracts non spécifiés	Aucune des deux bases ne les implémente	Retard si l'équipe les considère comme prioritaires sans le signaler tôt	Faible actuellement	Clarifier le calendrier (Phases 8-10, Section 26)	[À VALIDER]
PostgreSQL — deux bases séparées	Topologie de production non tranchée	Complexité opérationnelle si deux bases distinctes à maintenir	Moyenne	Décision explicite DevOps/équipe	[À VALIDER]
Divergence Java/IOTA lors d'une panne partielle	<code>iota-service</code> down pendant qu'un événement métier Java doit être ancré	Preuve manquante temporairement	Moyenne	Mécanisme de retry/outbox à étendre vers <code>iota-service</code> (pattern déjà connu côté Java)	[À VALIDER – conception]
Mauvais mapping métier/blockchain	Confondre un champ <code>iota_*</code> existant en base Java (jamais peuplé) avec une preuve réelle	Rapport erroné à un tiers/auditeur	Faible si ce document est diffusé et lu	Ce document + rappel systématique du statut [LOCAL JAVA] vs [EXISTANT TS]	—
Stockage comme troisième acteur non spécifié	Comportement de négociation du prix de stockage pas encore défini	Impact architecture marketplace inconnu à ce stade	Indéterminée	Attendre la clarification équipe avant tout impact sur le contrat IOTA	[À VALIDER – ne pas anticiper]

(Pas de score numérique utilisé, conformément à la consigne — les données ne le justifient pas.)

28. Open Questions

- Le `iota-connector` Java doit-il rester ? [À VALIDER]
- Doit-il devenir un orchestrateur/client du `iota-service` ? [À VALIDER]
- Quelle responsabilité exacte doit conserver `AncrageIotaHttp` (le remplacer, le désactiver définitivement, le garder en parallèle) ? [À VALIDER]
- Le `Ledger` Java reste-t-il le registre interne de référence ? [À VALIDER]
- Quelle preuve fait foi : ledger local ou preuve IOTA réelle ? [À VALIDER]
- Quelle est la source de vérité en cas de divergence ? [À VALIDER]
- Quel événement métier doit être ancré en premier (kWh vert ? certificat ? décision batterie) ? [À VALIDER]
- Où doit être stocké le `digest` IOTA côté Java ? [À VALIDER]
- Où doit être stocké un futur `objectId` (si des assets on-chain sont un jour introduits) ? [À VALIDER]
- Qui génère le `requestId` métier — Java ou `iota-service` ? (Réponse de code actuelle : toujours l'appelant, donc Java, si l'architecture cible est retenue) [EXISTANT dans le contrat TS, mais convention Java à formaliser]
- Qui conserve le `requestId` de bout en bout ? [À VALIDER]
- Qui est responsable de l'idempotence en cas d'échec partiel (Java a écrit, TS n'a jamais reçu, ou l'inverse) ? [À VALIDER]
- Qui possède les wallets (identité métier Java vs custody cryptographique TS) ? [À VALIDER]
- Le service IOTA possède-t-il la custody définitive des clés privées ? [À VALIDER]
- Quelle méthode d'authentification Java → TypeScript ? [À VALIDER]
- Où déployer `iota-service` (topologie réseau, environnement) ? [À VALIDER]
- Même PostgreSQL que le Backend ou base dédiée ? [À VALIDER]
- Quelle stratégie Devnet → Testnet → Mainnet, et à quel calendrier ? [À VALIDER]
- Quand commencer la tokenisation kWh (si jamais décidée on-chain) ? [À VALIDER]
- Quand implémenter les Move contracts (si jamais décidés) ? [À VALIDER]
- Les certificats doivent-ils être on-chain ? [À VALIDER]
- DID/W3C VC maintenant ou plus tard ? [À VALIDER]
- Comment gérer les retries de façon uniforme entre les deux systèmes ? [À VALIDER]
- Comment gérer une transaction IOTA réussie mais une réponse HTTP perdue côté Java ? [À VALIDER]
- Comment réconcilier PostgreSQL Java et PostgreSQL `iota-service` de façon régulière et automatisée ? [À VALIDER]
- Quelle stratégie pour le stockage comme troisième acteur de la marketplace ? [À VALIDER – sujet métier en cours de clarification, ne pas anticiper]
- Le pricing du stockage doit-il être géré par le même mécanisme de marketplace que producteur/consommateur ? [À VALIDER]
- Quel événement doit représenter la production énergétique validée, au sens d'un ancrage ? [À VALIDER]
- Où se trouve la source de vérité de `energy_kwh` (mesure brute, agrégation, prédiction) ? [À VALIDER]
- Comment distinguer mesure réelle, agrégation et prédiction dans un éventuel événement ancré ? [À VALIDER]

31. (Révélée par le code) Le format `requestId` Java (UUID avec tirets) est-il compatible avec la regex `^[A-Za-z0-9_-]{1,128}$` du service TS, ou faut-il une transformation ? [À VÉRIFIER]
32. (Révélée par le code) Le wallet store fichier de `iota-service` doit-il être remplacé avant tout test partagé avec le Backend, même en développement ? [À VALIDER]
33. (Révélée par le code) Qui porte la responsabilité opérationnelle de financer les wallets Devnet pour permettre un premier test de transfert réel ? [À VALIDER]

29. Validation Checklist

- ☐ Le responsable Backend Java a lu et validé `docs/integration-backend.md` (contrat REST existant)
- ☐ Décision d'architecture prise entre Options A/B/C pour le ledger Java (Section 12 de l'audit `iota-connector`)
- ☐ Mécanisme d'authentification Backend↔IOTA Service choisi et validé
- ☐ Convention `requestId` harmonisée entre Java (`RequestIds`) et TS (regex `^[A-Za-z0-9_-]{1,128}$`)
- ☐ Décision sur le remplacement du wallet store fichier de `iota-service` avant tout usage partagé
- ☐ Topologie PostgreSQL de production décidée (bases séparées ou partagées)
- ☐ Premier événement métier à ancrer identifié
- ☐ Emplacement de stockage du `digest` /référence IOTA côté Java décidé
- ☐ Stratégie de retry Java→TS définie et testée
- ☐ Calendrier Devnet→Testnet→Mainnet discuté avec l'équipe
- ☐ Sujet « stockage, troisième acteur marketplace » clarifié par l'équipe (indépendamment de l'architecture IOTA)
- ☐ Décision Git/repository prise (Section 25)
- ☐ Premier appel Java→IOTA Service réalisé en environnement de développement (preuve : digest réel obtenu)

30. Final Summary

Les deux travaux sont **complémentaires et non concurrents** : Hicham n'a pas refait le Backend Java, et le responsable Backend Java n'a pas besoin de refaire le service IOTA. Le Backend Java conserve toute sa logique métier (marketplace, wallets EUR, settlement, certificats, Digital Twin) ; le service IOTA isole la seule couche qui a besoin de connaître le SDK IOTA, un wallet cryptographique et un réseau distribué.

Ce qui est fait : une plateforme métier Java mature et un service IOTA TypeScript fonctionnel et testé sur son périmètre (réseau, wallet, transaction, idempotence, réconciliation).

Ce qui reste à faire : tout le raccordement entre les deux — authentification, premier appel réel, décision sur la source de vérité (registre local vs preuve IOTA), et toutes les questions listées en Section 28. Rien de cela n'est présenté comme réalisé dans ce document.

Prochaine étape concrète proposée [PROPOSITION] : une réunion courte entre Hicham et le responsable Backend Java pour parcourir ensemble la Validation Checklist (Section 29) et trancher au minimum les points 1 à 4, qui conditionnent tout le reste.

Fin du document. Document de travail — DRAFT / FOR REVIEW. Ne constitue pas une décision d'architecture finale.